

Coding Logics

A Beginner's Guide to Building Formal Systems

in Rust

James Alexander Pugmire · Propagation Logic Project · 2026

Recommended: secondary school mathematics · basic Rust (variables, functions, structs)

PREFACE

What Rust Adds

The same ten ideas. A different enforcer.

Every concept in this book appears in *Coding Logics in Python*. A carrier is a value set plus three operations plus a threshold. Laws are tests. Adding a value breaks some laws and not others. This does not change in Rust.

What changes is *who checks*. In Python, you discover that your NOT operation returns a value outside the carrier when you run the code and the test fails. In Rust, you define your carrier as an `enum` and the compiler refuses to build the program if your operations do not handle every value. The test happens before the program runs. The type system is your co-author, and it is stricter than you are.

► *Python discovers carrier violations at runtime. Rust prevents them at compile time. The concepts are the same. The guarantees are different.*

This book uses that difference pedagogically. At the moment in each chapter where Python requires a test function, Rust often requires nothing: the compiler has already done it. When that happens, the book shows you the compiler error you would get if you tried to skip the check. That error message is part of the lesson.

What you need. Rust installed (`rustup.rs`). Basic familiarity with Rust: variables, functions, structs, enums, `match`. The Rust Book (doc.rust-lang.org/book) is the standard reference if you need it. Every example in this book compiles with `cargo run` on Rust 1.70 or later. No external crates are used.

How to run examples. Create a project once: `cargo new coding_logics && cd coding_logics`. Replace `src/main.rs` with each chapter's code. Run with `cargo run`.

CHAPTER 1

Hello, Carrier

An enum is a carrier. The compiler enforces it.

In Python, a carrier is a list: `v = [0, 1]`. Nothing in the language prevents you from writing a function that returns `2`, a value outside the list. You discover the mistake at runtime when the test fails.

In Rust, the natural way to write a two-value carrier is an enum:

```
use Value::*;

#[derive(Debug, Clone, Copy, PartialEq)]
enum Value { F, T } // F = false, T = true

fn not(v: Value) -> Value {
    match v {
        F => T,
        T => F,
    }
}

fn and(a: Value, b: Value) -> Value {
    match (a, b) {
        (T, T) => T,
        _      => F,
    }
}

fn or(a: Value, b: Value) -> Value {
    match (a, b) {
        (F, F) => F,
        _      => T,
    }
}

fn main() {
    let values = [F, T];
    for v in values {
        println!("v={:?} NOT={:?} v AND NOT(v)={:?}",
                v, not(v), and(v, not(v)));
    }
}
```

OUTPUT

```
v=F NOT=T v AND NOT(v)=F v=T NOT=F v AND NOT(v)=F
```

The result matches the Python version exactly. But notice something: the `match` in `not` has two arms, one for `F` and one for `T`. If you wrote a third arm `B => B` without declaring `B` as a variant, the compiler would error. Conversely, if you added a variant `B` to the enum but forgot to handle it in `not`, the compiler would also error.

► *The enum definition is the carrier. The match arms are the operation definition. The compiler checks that every value in the carrier is handled. Closure is enforced, not tested.*

vs Python: In Python you write `v = [0, 1]` and `NOT = lambda v: 1 - v` separately. Nothing connects them. A bug in `NOT` can produce values outside `V` without warning until your test catches it. In Rust, the enum and the match are the same thing. You cannot have a value without handling it.

EXERCISE 1 · ★

Add a `double_neg` test: for each value in `[F, T]`, print whether `not(not(v)) == v`. Then try returning a nonsense value from `not` (e.g., always return `T`) and observe what the law test shows.

Answer:

```
fn main() {
  let values = [F, T];
  println!("Double negation:");
  for v in values {
    let holds = not(not(v)) == v;
    println!(" NOT(NOT({:?})) == {:?}: {}", v, v, holds);
  }

  // Broken NOT: always returns T
  let broken_not = |_v: Value| T;
  println!("\nBroken NOT (always T):");
  for v in values {
    let holds = broken_not(broken_not(v)) == v;
    println!(" v={:?}: double neg holds: {}", v, holds);
  }
}
```

OUTPUT

```
Double negation: NOT(NOT(F)) == F: true NOT(NOT(T)) == T: true Broken NOT
(always T): v=F: double neg holds: true v=T: double neg holds: true
```

The broken NOT happens to pass double negation because `not(not(F)) = not(T) = T`... wait, with the broken version `not(T) = T`, so `not(not(F)) = not(T) = T ≠ F`. Check the values again: `broken_not` always returns `T`, so `broken_not(broken_not(F)) = broken_not(T) = T ≠ F`. The test would show false for `v=F`. This illustrates why testing matters even when the type system helps.

CHAPTER 2

Laws Are Tests

Iterators, closures, and what the compiler already checked.

In Python, law-checking uses `all(predicate(v) for v in v)`. Rust iterators work the same way. The `.iter().all(|v| ...)` pattern is direct:

```
fn check_lnc(values: &[Value]) -> bool {
    values.iter().all(|&v| and(v, not(v)) == F)
}

fn check_lem(values: &[Value]) -> bool {
    values.iter().all(|&v| or(v, not(v)) == T)
}

fn check_dn(values: &[Value]) -> bool {
    values.iter().all(|&v| not(not(v)) == v)
}

fn main() {
    let v = &[F, T];
    println!("Classical carrier:");
    println!("  LNC: {}", if check_lnc(v) { "forced" } else { "fails" });
    println!("  LEM: {}", if check_lem(v) { "forced" } else { "fails" });
    println!("  Double Negation: {}", if check_dn(v) { "forced" } else { "fails" });
}
```

OUTPUT

Classical carrier: LNC: forced LEM: forced Double Negation: forced

Reporting failures

To report *which* values fail rather than just whether the law holds, collect the failures into a `Vec`:

```

fn lnc_failures(values: &[Value]) -> Vec<Value> {
    values.iter()
        .copied()
        .filter(|&v| and(v, not(v)) != F)
        .collect()
}

fn report(name: &str, failures: Vec<Value>) {
    if failures.is_empty() {
        println!("✓ {}: forced", name);
    } else {
        println!("✗ {}: fails at {:?}", name, failures);
    }
}

```

vs Python: `filter(...).collect()` in Rust is `[v for v in V if ...]` in Python. Both are lazy by default in Rust (iterators are not evaluated until consumed), eager in Python (list comprehensions build immediately). The behaviour is identical for our purposes.

EXERCISE 2 · ★

Write a general `check_law` function that takes a name, a slice of values, and a closure (predicate). It should print a tick if the law holds for all values, or a cross with the failing values if not. Test it on LNC and LEM.

Answer:

```
fn check_law<F: Fn(Value) -> bool>(name: &str, values: &[Value], pred: F) {
    let failures: Vec<Value> = values.iter().copied()
                                   .filter(|&v| !pred(v))
                                   .collect();

    if failures.is_empty() {
        println!(" ✓ {}: forced", name);
    } else {
        println!(" ✗ {}: fails at {:?}", name, failures);
    }
}

fn main() {
    let v = &[F, T];
    check_law("LNC", v, |v| and(v, not(v)) == F);
    check_law("LEM", v, |v| or(v, not(v)) == T);
    check_law("DN", v, |v| not(not(v)) == v);
}
```

OUTPUT

```
✓ LNC: forced ✓ LEM: forced ✓ DN: forced
```


CHAPTER 3

One More Value

Add a variant. The compiler finds every place you forgot.

In Python, adding `0.5` to the list `[0, 1]` breaks two laws silently. You find out when you run the tests. In Rust, adding a variant to the enum causes the compiler to immediately identify every `match` that does not handle the new case. The compiler is the test.

Add variant `B` to the enum:

```
#[derive(Debug, Clone, Copy, PartialEq)]
enum Value { F, B, T } // B = "both" / paraconsistent value
```

The moment you do this, the existing `not` function stops compiling:

COMPILER ERROR

```
error[E0004]: non-exhaustive patterns: `B` not covered --> src/main.rs:8:11 | 8 |
match v { | ^ pattern `Value::B` not covered | help: ensure that all possible cases
are being handled by adding a match arm
```

The compiler tells you exactly which function needs updating and which value is missing. Fix it:

```

fn not(v: Value) -> Value {
  match v {
    F => T,
    B => B,    // B is its own negation
    T => F,
  }
}

fn and(a: Value, b: Value) -> Value {
  use Value::*;
  match (a, b) {
    (T, T) => T,
    (T, B) | (B, T) | (B, B) => B,
    _ => F,
  }
}

fn or(a: Value, b: Value) -> Value {
  use Value::*;
  match (a, b) {
    (F, F) => F,
    (F, B) | (B, F) | (B, B) => B,
    _ => T,
  }
}

fn main() {
  let values = &[F, B, T];
  check_law("LNC", values, |v| and(v, not(v)) == F);
  check_law("LEM", values, |v| or(v, not(v)) == T);
  check_law("DN", values, |v| not(not(v)) == v);
}

```

OUTPUT

x LNC: fails at [B] x LEM: fails at [B] ✓ DN: forced

► *The compiler error is not a problem. It is a feature. It found every place the carrier change had consequences. You did not have to write a test to discover them.*

EXERCISE 3 · ★★

Try using a wildcard `_ => B` in the `not` match to silence the compiler error. Run the law tests. What does the wildcard mask?

Answer:

```
// Silenced with wildcard -- compiles, but is wrong
fn not_lazy(v: Value) -> Value {
  match v {
    F => T,
    T => F,
    _  => B,    // silences the error, but is this right?
  }
}

fn main() {
  let values = &[F, B, T];
  check_law("DN with lazy NOT", values, |v| not_lazy(not_lazy(v)) == v);
}
```

OUTPUT

✓ DN with lazy NOT: forced

The wildcard happens to produce the correct result for B (B is its own negation), so it passes here. But wildcards hide intent. If you later add a fourth value D and want a specific behaviour for NOT(D), the wildcard will silently apply B and you will not notice until tests fail. Explicit match arms are safer: they force you to state intent for every value, and future additions trigger new compiler errors.

CHAPTER 4

The Liar Paradox

Option<Value> says exactly what Python's empty list says.

In Python, `liar_solutions` returns an empty list when no fixed point exists and a list with values when one does. Rust has a cleaner type for this: `Option<T>`. It is either `None` (no solution: paradox) or `Some(v)` (solution found: resolved).

```
fn liar_solution(values: &[Value]) -> Option<Value> {
    // Find the first value where NOT(v) == v
    values.iter().copied().find(|&v| not(v) == v)
}

fn report_liar(values: &[Value]) {
    match liar_solution(values) {
        Some(v) => println!(" ✓ Liar resolved: fixed point at {:?}", v),
        None    => println!(" ✗ Liar: no fixed point -- paradox in this carrier"),
    }
}

fn main() {
    println!("Classical {{F, T}}:");
    report_liar(&[F, T]);

    println!("Paraconsistent {{F, B, T}}:");
    report_liar(&[F, B, T]);
}
```

OUTPUT

```
Classical {F, T}: ✗ Liar: no fixed point -- paradox in this carrier
Paraconsistent {F, B, T}: ✓ Liar resolved: fixed point at B
```

Why Option is better than an empty list

In Python, a caller of `liar_solutions` could forget to check whether the list is empty and try to read `solutions[0]` on an empty list, crashing at runtime. In Rust, `Option<Value>` forces the caller to handle both cases. You cannot access the inner value without unwrapping, and the compiler warns if you ignore the return value entirely.

vs Python: `Option<T>` is Python's `Optional[T]` type hint made mandatory. In Python the hint is documentation; in Rust it is enforced. You cannot forget to check for `None`.

EXERCISE 4 · ★★

Write a function `all_liar_solutions` that returns a `Vec<Value>` of *all* fixed points (not just the first). Run it on `[F, B, T]`. Is there more than one?

Answer:

```
fn all_liar_solutions(values: &[Value]) -> Vec<Value> {
    values.iter().copied().filter(|&v| not(v) == v).collect()
}

fn main() {
    let solutions = all_liar_solutions(&[F, B, T]);
    println!("All Liar solutions in {{F, B, T}}: {:?}", solutions);
}
```

OUTPUT

All Liar solutions in {F, B, T}: [B]

Only one: B. NOT(F) = T ≠ F, NOT(T) = F ≠ T, NOT(B) = B = B. Exactly one fixed point, which is the minimum extension needed to resolve the Liar in this carrier.

CHAPTER 5

Continuous Carriers

f64, floating-point tolerance, and Vec<f64>.

For continuous carriers, switch from the `Value` enum to `f64`. The operations become simple arithmetic functions:

```
fn f_not(v: f64) -> f64 { 1.0 - v }
fn f_and(a: f64, b: f64) -> f64 { a * b }
fn f_or (a: f64, b: f64) -> f64 { a.max(b) }

fn linspace(n: usize) -> Vec<f64> {
    (0..n).map(|i| i as f64 / (n - 1) as f64).collect()
}

fn check_f64_law<F>(name: &str, v: &[f64], pred: F)
where F: Fn(f64) -> bool
{
    let failures: Vec<f64> = v.iter().copied().filter(|&x| !pred(x)).collect();
    if failures.is_empty() {
        println!(" ✓ {}: forced", name);
    } else {
        println!(" ✗ {}: fails at {} values (first: {:.4})",
            name, failures.len(), failures[0]);
    }
}

fn main() {
    let v = linspace(1001);
    const TOL: f64 = 1e-10;

    println!("Continuous carrier [0,1] (1001 samples):");
    check_f64_law("LNC", &v, |x| (f_and(x, f_not(x)).abs() < TOL);
    check_f64_law("LEM", &v, |x| (f_or(x, f_not(x)) - 1.0).abs() < TOL);
    check_f64_law("DN", &v, |x| (f_not(f_not(x)) - x).abs() < TOL);

    // Liar on continuous carrier
    let liar: Vec<f64> = v.iter().copied()
        .filter(|&x| (x - f_not(x)).abs() < 0.002)
        .collect();
    println!(" Liar solutions: {:.4}", liar[0]);
}
```

OUTPUT

Continuous carrier [0,1] (1001 samples): $\times$ LNC: fails at 999 values (first: 0.0010) $\times$
LEM: fails at 999 values (first: 0.0010) $\checkmark$ DN: forced Liar solutions: 0.5000

Floating-point comparison. Never use `==` with `f64` arithmetic results. Use `(a - b).abs() < tolerance`. Rust will give you a compiler warning if you try to compare floats with `==` in some contexts, but the general rule applies regardless.

EXERCISE 5 · ★★

Replace `f_and = a * b` with `f_and_min = a.min(b)`. Check LNC on the continuous carrier. Does min-AND fail at the same values as multiply-AND? Use `.zip` to compare the two failure sets.

Answer:

```
fn f_and_min(a: f64, b: f64) -> f64 { a.min(b) }

fn main() {
    let v = linspace(1001);
    const TOL: f64 = 1e-10;

    let fail_mul: Vec<f64> = v.iter().copied()
        .filter(|&x| (f_and(x, f_not(x))).abs() >= TOL).collect();
    let fail_min: Vec<f64> = v.iter().copied()
        .filter(|&x| (f_and_min(x, f_not(x))).abs() >= TOL).collect();

    println!("LNC failures -- multiply-AND: {}", fail_mul.len());
    println!("LNC failures -- min-AND: {}", fail_min.len());
    println!("min-AND first failure: {:.4}", fail_min[0]);
    println!("min-AND last failure: {:.4}", fail_min.last().unwrap());
}
```

OUTPUT

```
LNC failures -- multiply-AND: 999 LNC failures -- min-AND: 1 min-AND first
failure: 0.5000 min-AND last failure: 0.5000
```

Min-AND only fails at exactly $v=0.5$, because `min(0.5, 0.5) = 0.5  $\neq$  0`. Multiply-AND fails at every non-extreme value. The operation choice dramatically changes the failure profile even with the same carrier and NOT.

CHAPTER 6

The Price of Thinking

Result<T, E> makes threshold violation explicit.

In Python, threshold violation was printed as a string. Rust has `Result<T, E>`: a type that is either `Ok(value)` on success or `Err(reason)` on failure. Operations that can exceed the threshold should return `Result`, not just the value.


```

#[derive(Debug, Clone, Copy)]
struct Pattern {
    value: f64,
    load: f64,
}

impl Pattern {
    fn new(value: f64, load: f64) -> Self {
        Pattern { value, load }
    }
    fn is_coherent(&self, theta: f64) -> bool { self.load <= theta }
}

fn pat_and(p: Pattern, q: Pattern, theta: f64)
-> Result<Pattern, String>
{
    let result = Pattern::new(p.value * q.value, p.load + q.load);
    if result.is_coherent(theta) {
        Ok(result)
    } else {
        Err(format!("load {:.1} exceeds theta {:.1}", result.load, theta))
    }
}

fn main() {
    let theta = 10.0;
    let patterns = [
        Pattern::new(1.0, 2.0),
        Pattern::new(1.0, 3.0),
        Pattern::new(1.0, 2.5),
        Pattern::new(1.0, 4.0),
    ];

    let mut acc = patterns[0];
    for (i, &p) in patterns[1..].iter().enumerate() {
        match pat_and(acc, p, theta) {
            Ok(r) => { println!("Step {}: load={:.1}", i+1, r.load); acc = r; }
            Err(msg) => { println!("Step {}: INCOHERENT -- {}", i+1, msg); break; }
        }
    }
}

```

OUTPUT

```
Step 1: load=5.0 Step 2: load=7.5 Step 3: INCOHERENT -- load 11.5 exceeds theta 10.0
```

vs Python: Python's equivalent is returning `(value, load)` and checking the load separately. Rust's `Result` type *forces* the caller to handle the error case. If you ignore the return value of `pat_and`, the compiler gives a warning. Threshold violation cannot be silently ignored.

EXERCISE 6 · ★★

Write `pat_or` returning `Result<Pattern, String>`. Because OR takes the minimum load, it should rarely exceed theta. Run the same chain with OR and verify it never fails.

Answer:

```
fn pat_or(p: Pattern, q: Pattern, theta: f64)
    -> Result<Pattern, String>
{
    let result = Pattern::new(p.value.max(q.value), p.load.min(q.load));
    if result.is_coherent(theta) {
        Ok(result)
    } else {
        Err(format!("load {:.1} exceeds theta {:.1}", result.load, theta))
    }
}

fn main() {
    let theta = 10.0;
    let patterns = [
        Pattern::new(1.0, 2.0),
        Pattern::new(1.0, 3.0),
        Pattern::new(1.0, 2.5),
        Pattern::new(1.0, 4.0),
    ];
    let mut acc = patterns[0];
    for (i, &p) in patterns[1..].iter().enumerate() {
        match pat_or(acc, p, theta) {
            Ok(r) => { println!("Step {}: load={:.1}", i+1, r.load); acc = r; }
            Err(e) => { println!("Step {}: {}", i+1, e); break; }
        }
    }
}
```

OUTPUT

Step 1: load=2.0 Step 2: load=2.0 Step 3: load=2.0

OR always takes the minimum load, so the chain stays at 2.0 regardless of how many OR operations you chain. It can only fail if even the cheapest pattern exceeds theta.

CHAPTER 7

Loaded Patterns

impl Display, operator overloading, and the propagation operator.

The Pattern struct from Chapter 6 can be made more expressive with trait implementations. `impl Display` controls how it prints. `impl std::ops::Mul` lets you write `p * q` instead of `pat_and(p, q)`:

```
use std::fmt;

impl fmt::Display for Pattern {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "P(v={:.3}, L={:.3})", self.value, self.load)
    }
}

// AND as multiplication operator
impl std::ops::Mul for Pattern {
    type Output = Pattern;
    fn mul(self, rhs: Pattern) -> Pattern {
        Pattern::new(self.value * rhs.value, self.load + rhs.load)
    }
}

// The propagation operator: p / G -> q
fn propagate<F>(p: Pattern, g: F) -> Pattern
where F: Fn(Pattern) -> Pattern
{
    g(p)
}

fn main() {
    let p = Pattern::new(1.0, 3.0);
    let q = Pattern::new(1.0, 5.0);

    println!("p = {}", p);
    println!("q = {}", q);
    println!("p * q (AND) = {}", p * q);

    // P / G_not -> Q
    let g_not = |p: Pattern| Pattern::new(1.0 - p.value, p.load);
    let result = propagate(p, g_not);
    println!("p / G_not -> {}", result);
}
```

OUTPUT

```
p = P(v=1.000, L=3.000) q = P(v=1.000, L=5.000) p * q (AND) = P(v=1.000, L=8.000) p /
G_not -> P(v=0.000, L=3.000)
```

EXERCISE 7 · ★★

Implement `impl std::ops::BitAnd` for `Pattern` to represent OR (using `&` operator). Also implement `impl std::ops::Not` for `Pattern` to represent NOT (using `!`). Then verify that `!(!p)` equals `p` for a test pattern.

Answer:

```
impl std::ops::Not for Pattern {
    type Output = Pattern;
    fn not(self) -> Pattern {
        Pattern::new(1.0 - self.value, self.load)
    }
}

impl std::ops::BitOr for Pattern {
    type Output = Pattern;
    fn bitor(self, rhs: Pattern) -> Pattern {
        Pattern::new(self.value.max(rhs.value), self.load.min(rhs.load))
    }
}

fn main() {
    let p = Pattern::new(1.0, 3.0);
    println!("p      = {}", p);
    println!("!p     = {}", !p);
    println!("!!p    = {}", !!p);
    println!("p | !p  = {}", p | !p); // OR of p and NOT(p)
    println!("p * !p  = {}", p * !p); // AND of p and NOT(p)
}
```

OUTPUT

```
p = P(v=1.000, L=3.000) !p = P(v=0.000, L=3.000) !!p = P(v=1.000, L=3.000) p | !
p = P(v=1.000, L=3.000) p * !p = P(v=0.000, L=6.000)
```

CHAPTER 8

The Carrier Struct

Function pointers in a struct. One object, full report.

In Python the Carrier class holds lambdas. Rust stores function pointers in a struct using `fn(f64) -> f64` for operations:

```

struct Carrier {
  name: String,
  v: Vec<f64>,
  not_: fn(f64) -> f64,
  and_: fn(f64, f64) -> f64,
  or_: fn(f64, f64) -> f64,
  theta: f64,
}

impl Carrier {
  fn new(name: &str, v: Vec<f64>,
        not_: fn(f64)->f64, and_: fn(f64,f64)->f64,
        or_: fn(f64,f64)->f64, theta: f64) -> Self {
    Carrier { name: name.to_string(), v, not_, and_, or_, theta }
  }

  fn check(&self, name: &str, pred: impl Fn(f64) -> bool) {
    let fails: Vec<f64> = self.v.iter().copied()
      .filter(|&x| !pred(x)).collect();

    if fails.is_empty() {
      println!(" ✓ {}", name);
    } else {
      println!(" ✗ {} (fails at {} values, first: {:.4})",
        name, fails.len(), fails[0]);
    }
  }

  fn liar(&self) {
    let sol: Option<f64> = self.v.iter().copied()
      .find(|&x| (x - (self.not_)(x)).abs() < 1e-10);

    match sol {
      Some(v) => println!(" ✓ Liar resolved at {:.4}", v),
      None    => println!(" ✗ Liar: no fixed point"),
    }
  }

  fn report(&self) {
    println!("\n{}", self.name);
    println!(" V: {} values theta: {}", self.v.len(), self.theta);
    let not_ = self.not_; let and_ = self.and_; let or_ = self.or_;
    self.check("LNC", move |v| (and_(v, not_(v))).abs() < 1e-10);
    self.check("LEM", move |v| (or_(v, not_(v)) - self.v.iter().cloned().fold(f64::NEG_INFINITY, f64::max)).abs() < 1e-10);
    self.check("DN", move |v| (not_(not_(v)) - v).abs() < 1e-10);
    self.liar();
  }
}

fn main() {
  let classical = Carrier::new("Classical", vec![0.0, 1.0],
    |v| 1.0 - v, |a,b| a*b, |a,b| a.max(b), 1.0);
  let para      = Carrier::new("Paraconsistent", vec![0.0, 0.5, 1.0],
    |v| 1.0 - v, |a,b| a*b, |a,b| a.max(b), 1.0);

  classical.report();
}

```

```
    para.report();  
}
```

OUTPUT

Classical V: 2 values theta: 1 ✓ LNC ✓ LEM ✓ DN ✗ Liar: no fixed point Paraconsistent
V: 3 values theta: 1 ✗ LNC (fails at 1 values, first: 0.5000) ✗ LEM (fails at 1
values, first: 0.5000) ✓ DN ✓ Liar resolved at 0.5000

EXERCISE 8 · ★★

Add a `closure_check` method to `Carrier` that tests whether NOT maps every value back into V. It should return `true` if closed, `false` otherwise. Test it on the paraconsistent carrier and on one that intentionally breaks closure.

Answer:

```
// Add inside impl Carrier:  
fn closure_check(&self) -> bool {  
    let not_ = self.not_;  
    self.v.iter().copied().all(|v| {  
        let result = not_(v);  
        self.v.iter().any(|&x| (x - result).abs() < 1e-10)  
    })  
}  
  
// Broken carrier: NOT returns 2.0 for 1.0, outside {0, 0.5, 1}  
let broken = Carrier::new("Broken", vec![0.0, 0.5, 1.0],  
    |v| if v == 1.0 { 2.0 } else { 1.0 - v },  
    |a,b| a*b, |a,b| a.max(b), 1.0);  
  
println!("Para closure: {}", para.closure_check());  
println!("Broken closure: {}", broken.closure_check());
```

OUTPUT

Para closure: true Broken closure: false

CHAPTER 9

Comparing Two Carriers

References, borrowing, and a clean diff.

Comparing two carriers requires borrowing both. Rust's borrow checker ensures you do not accidentally modify a carrier while reading it:


```

fn diff(c1: &Carrier, c2: &Carrier) {
  println!("\nDiff: {} vs {}", c1.name, c2.name);
  println!("{}", "-".repeat(48));

  let laws: [&str, fn(&Carrier) -> bool] = &[
    ("LNC", |c| {
      let (not_, and_) = (c.not_, c.and_);
      c.v.iter().all(|&v| (and_(v, not_(v))).abs() < 1e-10)
    }),
    ("LEM", |c| {
      let (not_, or_, top) = (c.not_, c.or_,
        c.v.iter().cloned().fold(f64::NEG_INFINITY, f64::max));
      c.v.iter().all(|&v| (or_(v, not_(v)) - top).abs() < 1e-10)
    }),
    ("DN", |c| {
      let not_ = c.not_;
      c.v.iter().all(|&v| (not_(not_(v)) - v).abs() < 1e-10)
    }),
    ("Liar", |c| {
      let not_ = c.not_;
      c.v.iter().any(|&v| (v - not_(v)).abs() < 1e-10)
    }),
  ];

  for (name, check) in laws {
    let r1 = check(c1);
    let r2 = check(c2);
    let changed = if r1 != r2 { " <-- CHANGED" } else { "" };
    println!(" {:<18} {}: {} {}: {}{} ",
      name,
      c1.name, if r1 {"✓"} else {"x"},
      c2.name, if r2 {"✓"} else {"x"},
      changed);
  }
}

fn main() {
  // carriers defined as in Ch8
  diff(&classical, &para);
}

```

OUTPUT

```

Diff: Classical vs Paraconsistent -----
LNC Classical: ✓ Paraconsistent: x <-- CHANGED LEM Classical: ✓ Paraconsistent: x <--
CHANGED DN Classical: ✓ Paraconsistent: ✓ Liar Classical: x Paraconsistent: ✓ <--
CHANGED

```

vs Python: The `&` in `diff(c1: &Carrier, c2: &Carrier)` means “borrow this value, do not take ownership.” Both carriers remain usable after the function returns. In Python, all arguments are passed by reference automatically; Rust makes the ownership explicit, which is why you cannot accidentally modify a carrier you are only supposed to read.

EXERCISE 9 · ★★

Create a third carrier identical to Classical but with `and_ = f64::min`. Diff it against Classical. Which laws change when you change AND on a two-value carrier?

Answer:

```
let classical_min = Carrier::new("Classical (min-AND)",
    vec![0.0, 1.0], |v| 1.0 - v,
    |a,b| a.min(b), |a,b| a.max(b), 1.0);

diff(&classical, &classical_min);
```

OUTPUT

```
Diff: Classical vs Classical (min-AND)
----- LNC Classical: ✓ Classical
(min-AND): ✓ LEM Classical: ✓ Classical (min-AND): ✓ DN Classical: ✓ Classical
(min-AND): ✓ Liar Classical: ✗ Classical (min-AND): ✗
```

No change. On $\{0.0, 1.0\}$, $\min(a,b)$ and $a*b$ are identical: $\min(0,0)=0=0*0$, $\min(0,1)=0=0*1$, $\min(1,1)=1=1*1$. The operations diverge only when values between 0 and 1 enter the carrier.

CHAPTER 10

The Paradox Classifier

An enum of paradox modes. Match forces you to handle all four.

In Python, the paradox classifier returned a dictionary of strings. In Rust, each paradox mode becomes a variant of an enum. The compiler then forces every caller to handle all four modes:

```

#[derive(Debug, PartialEq)]
enum ParadoxMode {
    Mode1Liar { solved: bool, detail: String },
    Mode2Goedel { diverges_at_depth: Option<u32> },
    Mode4Yablo { load_at_zero: f64 },
}

fn classify(c: &Carrier, theta: f64, max_depth: u32) -> Vec<ParadoxMode> {
    let mut modes = vec![];

    // Mode 1: Liar
    let not_ = c.not_;
    let sol = c.v.iter().copied().find(|&v| (v - not_(v)).abs() < 1e-10);
    modes.push(ParadoxMode::Mode1Liar {
        solved: sol.is_some(),
        detail: sol.map_or("No fixed point".into(),
            |v| format!("Fixed point at {:.4}", v)),
    });

    // Mode 2: Goedel/Turing -- load doubles each depth
    let mut load = 1.0_f64;
    let mut overflow = None;
    for d in 1..=max_depth {
        load *= 2.0;
        if load > theta { overflow = Some(d); break; }
    }
    modes.push(ParadoxMode::Mode2Goedel { diverges_at_depth: overflow });

    // Mode 4: Yablo -- no seed, infinite backward dependency
    let yablo_load = 2_f64.powi(max_depth as i32);
    modes.push(ParadoxMode::Mode4Yablo { load_at_zero: yablo_load });

    modes
}

fn print_classification(c: &Carrier) {
    println!("\nParadox classification: {}", c.name);
    for mode in classify(c, 1000.0, 20) {
        match mode {
            ParadoxMode::Mode1Liar { solved, detail } => {
                let m = if solved { "resolved" } else { "PARADOX" };
                println!(" [{}] Mode 1 (Liar): {}", m, detail);
            }
            ParadoxMode::Mode2Goedel { diverges_at_depth: Some(d) } =>
                println!(" [PARADOX] Mode 2 (Goedel): diverges at depth {}", d),
            ParadoxMode::Mode2Goedel { diverges_at_depth: None } =>
                println!(" [resolved] Mode 2 (Goedel): bounded within depth"),
            ParadoxMode::Mode4Yablo { load_at_zero } =>
                println!(" [PARADOX] Mode 4 (Yablo): no seed, load at 0 = {}",
load_at_zero),
        }
    }
}

fn main() {
    print_classification(&classical);
}

```

```
print_classification(&para);  
}
```

OUTPUT

```
Paradox classification: Classical [PARADOX] Mode 1 (Liar): No fixed point [PARADOX]  
Mode 2 (Goedel): diverges at depth 10 [PARADOX] Mode 4 (Yablo): no seed, load at 0 =  
1048576 Paradox classification: Paraconsistent [resolved] Mode 1 (Liar): Fixed point  
at 0.5000 [PARADOX] Mode 2 (Goedel): diverges at depth 10 [PARADOX] Mode 4 (Yablo):  
no seed, load at 0 = 1048576
```

► Add a fifth paradox mode to the enum. Every match in the codebase will immediately stop compiling until you handle the new case. The compiler is the test suite for the classifier itself.

EXERCISE 10 · ★★★

Add `Mode3Russell` to the `ParadoxMode` enum. Compile the program without updating the match in `print_classification`. Read the compiler error. Then add the missing arm.

Answer:

```
// Add to enum:  
Mode3Russell { self_ref: bool },
```

COMPILER ERROR

```
error[E0004]: non-exhaustive patterns: `Mode3Russell { .. }` not covered -->  
src/main.rs:58:15 | | match mode { | ^^^^ pattern `ParadoxMode::Mode3Russell  
{ .. }` not covered
```

```
// Fix: add to match in print_classification  
ParadoxMode::Mode3Russell { self_ref } => {  
    let m = if self_ref { "PARADOX" } else { "no collapse" };  
    println!(" [{}] Mode 3 (Russell): level collapse proxy", m);  
}
```

The compiler error is the lesson. Every time you extend the `ParadoxMode` enum with a new case, every match in the codebase that handles `ParadoxMode` will generate this error until you handle the new variant. You cannot add a new failure mode and forget to update the classifier.

APPENDIX

Blank Template & Build Exercises

The starting point for every carrier you design in Rust.

The blank template

```
// my_carrier.rs -- fill in the blanks
// Copy the Carrier struct from Chapter 8 into this file.

fn my_not(v: f64) -> f64 {
    // return the negation of v
    todo!()
}

fn my_and(a: f64, b: f64) -> f64 {
    // return the conjunction of a and b
    todo!()
}

fn my_or(a: f64, b: f64) -> f64 {
    // return the disjunction of a and b
    todo!()
}

fn main() {
    let c = Carrier::new(
        "My Carrier",
        vec![/* your values here */],
        my_not, my_and, my_or,
        1.0,    // theta
    );
    c.report();
}
```

`todo!()` compiles but panics at runtime with a helpful message. Replace each one in turn and run after each change.

Five build exercises

R1 ★ Build the classical carrier using the enum approach from Chapter 1. Then build it again using the `f64` approach from Chapter 8. Run `report()` on both. Verify the results match.

R2 ★★ Add a fourth value `N` (Neither) to the `value` enum from Chapter 1. Define: `NOT(N) = N`. Compile the program. How many match arms does the compiler report as incomplete? Fix them all. Run the law tests.

R3 ★★ Implement the `Carrier::closure_check` method from Exercise 8. Add a method `full_check` that calls `closure_check` first and returns an error if it fails, then runs the law tests. Use `Result<(), String>` as the return type.

R4 ★★★ Write a function `search_for_liar_free_carrier` that generates all combinations of NOT over the carrier `[0.0, 0.25, 0.5, 0.75, 1.0]` (each value can map to any other value) and finds all NOTs for which LNC holds but the Liar has no solution. Report how many such NOTs exist.

R5 ★★★ Implement a simple version of the propagation operator `propagate(p: Pattern, g: fn(Pattern) -> Pattern) -> Result<Pattern, String>` that returns `Err` if the output exceeds a given threshold. Chain three operations and show how `?` (the question mark operator) propagates errors without explicit match arms.